TP Rust embarqué : le feu de circulation

IA03 — UTT — Explication du code, simplement · Mehdi Ez-Zouak, Paul Louis Ledoux, Clément Menaucourt

> **En une phrase :** on écrit un petit programme en Rust, on l'envoie sur une carte électronique, et ce programme allume 3 LEDs comme un feu de circulation. Quand on appuie sur le bouton bleu, le feu passe en panne : le jaune clignote.

1. Le décor : qu'est-ce qu'on manipule ?

La carte (STM32 Nucleo-H753ZI) est un mini-ordinateur très simple, un *microcontrôleur*. Pas d'écran, pas de Windows ni de Linux dessus : il exécute un seul programme, le nôtre, en boucle, tant qu'il est alimenté.

Sur la carte, il y a 3 LEDs et un bouton bleu. Chacun est relié à une **broche** (un « fil ») du microcontrôleur. Les broches ont des noms du style **PB0** = port B, broche numéro 0.

Élément	Broche	Ce qu'on en fait
LED verte	PB0	Sortie : on met la broche à « haut » pour allumer, « bas » pour éteindre
LED jaune	PE1	Sortie, pareil
LED rouge	PB14	Sortie, pareil
Bouton bleu	PC13	Entrée : on lit « haut » si on appuie, « bas » sinon

Comment le programme arrive sur la carte

- On écrit le code sur l'ordinateur (fichier **src/main.rs**).
- On tape **cargo run** : Rust compile le code pour le processeur de la carte (un ARM, pas un processeur de PC).
- L'outil **probe-rs** envoie le programme sur la carte par le câble USB (on dit « flasher »).
- Le programme démarre. Les **println!** s'affichent dans le terminal du PC : c'est notre seul moyen de voir ce qui se passe (on appelle ça la console RTT). Ctrl+C pour quitter.

Pourquoi le code a l'air bizarre au début

Comme il n'y a pas de système d'exploitation, le programme doit tout faire lui-même. Les premières lignes du fichier le disent à Rust :

```
#![no_std]   // pas de bibliothèque standard (pas de fichiers, pas d'écran...)
#![no_main]  // pas de main() classique : c'est cortex_m_rt::entry qui démarre
```

Et **fn main() -> !** : le **!** veut dire « cette fonction ne s'arrête jamais ». Normal : un feu de circulation ne s'éteint pas tout seul.

2. Partie 1 : le feu avec une boucle

Le prof nous donnait un modèle qui faisait clignoter les 3 LEDs en même temps. On l'a transformé en vrai feu : **vert 3 s → jaune 1 s → rouge 3 s**, et on recommence. Si on maintient le bouton : le jaune clignote. Quand on lâche : on repart au rouge.

Étape 1 — Préparer les LEDs et le bouton

```
let mut led_green  = gpiob.pb0.into_push_pull_output();   // PB0 en sortie
let mut led_yellow = gpioe.pe1.into_push_pull_output();   // PE1 en sortie
let mut led_red    = gpiob.pb14.into_push_pull_output();  // PB14 en sortie
let bouton = gpioc.pc13.into_floating_input();            // PC13 en entrée
```

On dit simplement à chaque broche si elle sert à **écrire** (LED) ou à **lire** (bouton). Le mot **mut** veut dire « modifiable » : en Rust, une variable est figée par défaut. Les LEDs changent d'état, donc elles sont **mut**. Le bouton, on ne fait que le lire.

Étape 2 — Décrire les 3 couleurs avec un enum

```
enum Feu { Vert, Jaune, Rouge }

fn duree_ms(self) -> u32 {          // combien de temps dure chaque couleur
    match self {
        Feu::Vert  => 3000,
        Feu::Jaune => 1000,
        Feu::Rouge => 3000,
    }
}

fn suivant(self) -> Feu {           // quelle couleur vient après
    match self {
        Feu::Vert  => Feu::Jaune,
        Feu::Jaune => Feu::Rouge,
        Feu::Rouge => Feu::Vert,
    }
}
```

Un **enum** = une liste de valeurs possibles. Notre feu est soit Vert, soit Jaune, soit Rouge, rien d'autre. Le **match**, c'est un « si c'est ça, alors ça » qui oblige à traiter tous les cas (si on oublie une couleur, Rust refuse de compiler).

Étape 3 — La boucle principale

L'idée clé : au lieu d'attendre 3 secondes d'un coup (pendant lesquelles on ne verrait pas le bouton), on attend **10 ms à la fois**, et entre chaque petite attente on regarde le bouton. On compte le temps écoulé dans **ecoule_ms**.

```
loop {
    if bouton.is_high() {                       // bouton appuyé ?
        // -> mode panne : vert et rouge éteints, jaune qui clignote
        if (ecoule_ms / 500) % 2 == 0 { led_yellow.set_high(); }
        else                          { led_yellow.set_low();  }
    } else {
        if ecoule_ms >= feu.duree_ms() {        // la couleur a assez duré ?
            feu = feu.suivant();                // on passe à la suivante
```

```
            ecoule_ms = 0;                      // et on remet le chrono à 0
        }
        // on allume la LED de la couleur actuelle, on éteint les autres
    }
    delay.delay_ms(10);                         // petite pause de 10 ms
    ecoule_ms += 10;                            // le chrono avance
}
```

> **Le clignotement en une ligne : (ecoule_ms / 500) % 2** vaut 0 pendant 500 ms, puis 1 pendant 500 ms, puis 0... (**%** = reste de la division). Donc le jaune est allumé une demi-seconde sur deux.

Cette façon de faire (regarder le bouton nous-mêmes, très souvent) s'appelle le **polling**.

Le bug qu'on a trouvé (et corrigé)

Au premier essai, le feu restait bloqué au rouge. En fait il n'était pas bloqué : **tout allait 8 fois trop lentement** (le rouge durait 24 s au lieu de 3 s).

- La fonction d'attente fournie (**Delay** de la bibliothèque stm32h7xx-hal) est écrite pour une ancienne version de la puce.
- Nos cartes ont une version plus récente (« révision V ») où l'horloge utilisée est divisée par 8.
- Correction : on utilise le **Delay** d'une autre bibliothèque (**cortex_m**), qui se cale sur l'horloge du processeur et donne le bon temps sur toutes les cartes.

```
use cortex_m::delay::Delay;
let mut delay = Delay::new(cp.SYST, ccdr.clocks.c_ck().raw());
```

Même le modèle du prof a ce problème sur nos cartes : ses LEDs clignotent toutes les 4 s au lieu de 0,5 s.

3. Partie 2 : le bouton avec une interruption

Le problème du polling, avec une image

> **Polling** = aller ouvrir la porte toutes les 10 secondes pour voir si quelqu'un est là. Si on est occupé ailleurs, le visiteur attend.
> **Interruption** = installer une **sonnette**. On fait ce qu'on veut, et quand ça sonne, on s'arrête, on va ouvrir, puis on reprend exactement ce qu'on faisait.

Dans la partie 2, le bouton devient une sonnette : c'est le matériel de la carte qui prévient le processeur dès que le bouton change, même s'il est au milieu d'une attente de 3 secondes.

RTIC : le programme découpé en 3 morceaux

Morceau	Quand il tourne	Ce qu'il fait chez nous
init	Une seule fois, au démarrage	Prépare les LEDs, le bouton et la « sonnette »

idle	Tout le temps, en boucle	Fait tourner le feu (vert, jaune, rouge ou clignotement)
btn_isr	Seulement quand le bouton change	Note si le bouton est appuyé ou relâché

RTIC est la bibliothèque qui organise ce découpage. Elle ressemble à Arduino : **init** = setup(), **idle** = loop(), et **btn_isr** en plus pour la sonnette.

Le carnet partagé : light_failed

btn_isr et **idle** doivent se parler. Ils partagent une seule variable vrai/faux : **light_failed** (« le feu est en panne »). btn_isr l'écrit, idle la lit.

Pour y toucher, il faut passer par **.lock(...)** :

```
let failed = cx.shared.light_failed.lock(|lf| *lf);   // lire (dans idle)
cx.shared.light_failed.lock(|lf| *lf = appuye);       // écrire (dans btn_isr)
```

> **Pourquoi lock ?** Imaginez deux personnes qui écrivent dans le même carnet en même temps : ça donne n'importe quoi. Le **lock**, c'est « je prends le carnet, personne d'autre n'y touche, je le rends ». Rust refuse de compiler si on oublie le lock.

Les autres choses ne sont pas partagées (**local**) : les LEDs appartiennent à idle, le bouton appartient à btn_isr. C'est pour ça qu'idle ne peut plus lire le bouton directement : il lit le carnet.

Ce qu'on a ajouté dans init : installer la sonnette

```
let mut btn = gpioc.pc13.into_input();                    // 1. PC13 en entrée
btn.make_interrupt_source(&mut dp.SYSCFG);                // 2. sonnette sur PC13
btn.trigger_on_edge(&mut dp.EXTI, Edge::RisingFalling);   // 3. appui ET relâchement
btn.enable_interrupt(&mut dp.EXTI);                       // 4. allumer la sonnette
```

- **EXTI** = la pièce de la carte qui gère les sonnettes (External Interrupt).
- **Front montant** (Rising) = la tension monte = on **appuie**. **Front descendant** (Falling) = la tension descend = on **relâche**.
- On veut les **deux** (RisingFalling) : sinon on saurait quand on appuie, mais jamais quand on lâche, et le feu resterait en panne pour toujours.

Ce qu'on a ajouté : btn_isr, la fonction appelée quand ça sonne

```
#[task(binds = EXTI15_10, local = [btn], shared = [light_failed])]
fn btn_isr(mut cx: btn_isr::Context) {
    let appuye = cx.local.btn.is_high();              // a) appuyé ou relâché ?
    cx.shared.light_failed.lock(|lf| *lf = appuye);  // b) on l'écrit dans le carnet
    cx.local.btn.clear_interrupt_pending_bit();       // c) on éteint la sonnette
    println!("ISR : ...");                            // d) message dans la console
}
```

- **binds = EXTI15_10** : « appelle cette fonction quand la sonnette n°13 sonne ». Les broches 10 à 15 partagent la même sonnette, qui s'appelle EXTI15_10. PC13 = numéro 13, donc c'est elle.

- **a)** La sonnette dit juste « ça a changé ». On relit le bouton pour savoir dans quel sens.
- **b)** On met à jour le carnet : vrai = panne, faux = normal.
- **c) Très important** : si on n'éteint pas la sonnette, elle continue de sonner, la fonction est rappelée à l'infini et la carte a l'air gelée.
- **d)** Un message pour vérifier dans le terminal que ça marche.

Ce qu'on a ajouté : idle, la boucle du feu

```
#[idle(local = [delay, led_green, led_yellow, led_red], shared = [light_failed])]
fn idle(mut cx: idle::Context) -> ! {
    let mut etape = 0;                                  // 0 vert, 1 jaune, 2 rouge
    loop {
        let failed = cx.shared.light_failed.lock(|lf| *lf);  // on lit le carnet
        if failed {
            // jaune allumé 500 ms, éteint 500 ms
        } else if etape == 0 {
            // vert allumé 3 s, puis etape = 1
        } else if etape == 1 {
            // jaune allumé 1 s, puis etape = 2
        } else {
            // rouge allumé 3 s, puis etape = 0
        }
    }
}
```

La première ligne liste ce dont idle a besoin : les 3 LEDs et le delay (à lui tout seul), et le carnet light_failed (partagé). À chaque tour, il regarde le carnet et affiche la bonne couleur.

> **Ce qu'on observe (et c'est normal) :** quand on appuie, le message « ISR » apparaît tout de suite (la sonnette est instantanée). Mais la LED peut mettre jusqu'à 3 s à changer, parce qu'idle est peut-être en train d'attendre la fin du vert. Il ne relit le carnet qu'après. L'énoncé le dit aussi.

4. Mini-lexique

Mot	En clair
Microcontrôleur	Mini-ordinateur sur une puce, qui exécute un seul programme
Broche / GPIO	Un « fil » de la puce qu'on peut mettre à haut/bas (sortie) ou lire (entrée)
Flasher	Envoyer le programme dans la mémoire de la carte (cargo run le fait)
Polling	Vérifier soi-même, très souvent, si quelque chose a changé
Interruption	Le matériel prévient le processeur tout seul (la sonnette)
EXTI	La pièce de la carte qui gère les interruptions venant des broches
Front montant / descendant	Le moment où la tension monte (appui) / descend (relâchement)
Handler / ISR	La fonction appelée quand l'interruption arrive (ici btn_isr)
RTIC	Bibliothèque Rust qui organise init / idle / interruptions
Ressource partagée	Variable utilisée par plusieurs morceaux du programme (light_failed)
lock	Prendre la variable partagée pour soi le temps de la lire ou l'écrire
println! / RTT	Messages envoyés de la carte vers le terminal du PC

5. Si le prof pose une question

Question	Réponse courte
Pourquoi une interruption plutôt que du polling ?	Le bouton est détecté instantanément, même pendant une attente, sans gaspiller le processeur à vérifier en boucle.
Pourquoi RisingFalling ?	Il faut savoir quand on appuie (montant) ET quand on relâche (descendant).
Pourquoi EXTI15_10 ?	PC13 est sur la ligne EXTI 13, et les lignes 10 à 15 partagent ce vecteur.
À quoi sert clear_interrupt_pending_bit ?	À dire « interruption traitée ». Sans ça, elle se redéclenche sans fin et la carte gèle.
Pourquoi lock ?	light_failed est partagé entre deux tâches : le lock évite qu'elles y touchent en même temps.
Pourquoi la LED ne change pas tout de suite ?	idle est bloqué dans un delay_ms ; il ne relit light_failed qu'à la fin de l'attente.
Pourquoi vous n'utilisez pas le Delay du template en partie 1 ?	Il est prévu pour la révision Y de la puce ; sur nos cartes (révision V) il est 8 fois trop lent.